

Capstone Project

Automated Application Delivery on AWS Using Jenkins, Terraform, Ansible, and ECS

Introduction

Modern application deployment involves several stages, including building, testing, security scanning, deployment, and monitoring. Automating these stages helps create a consistent and streamlined delivery process.

In this project, a containerized web application is deployed on AWS through a **Jenkins CI/CD pipeline**. **Terraform** is used to provision the AWS infrastructure, while **Ansible** is used to configure the DevOps EC2 server. Jenkins handles the application dependency installation, linting, and build, performs code analysis using **SonarQube**, scans the Docker image using **Trivy**, and pushes the Docker image to **Amazon ECR**.

The application is deployed to **Amazon ECS** using a **Blue/Green deployment strategy** with an **Application Load Balancer** and **separate target groups** for the application environments. **Amazon CloudWatch** is used for logs and metrics, while **Grafana** provides monitoring dashboards. This project brings together the AWS and DevOps concepts covered throughout the course into a single automated workflow.

Technology Stack

Terraform	Infrastructure Provisioning
Ansible	Server Configuration
Git, AWS CodeCommit	Source Code Management
Jenkins	Pipeline Automation
Amazon EC2	DevOps Server
Docker	Application Containerization
Amazon ECR	Image Storage
Amazon ECS	Container Deployment
Application Load Balancer	Traffic Management

SonarQube	Code Quality and Security Analysis
Trivy	Container Vulnerability Scanning
Amazon CloudWatch	Logs and Metrics
Grafana	Monitoring Dashboards
Amazon VPC	Network Infrastructure
Internet Gateway	Internet Connectivity
NAT Gateway	Outbound Internet Access
AWS IAM	Access Management

Project Architecture

Infrastructure Layer

- **Terraform** provisions the AWS infrastructure required for the project.
- The infrastructure includes an **Amazon VPC**, two public subnets across separate Availability Zones, two private subnets, security groups, IAM roles, an **EC2 DevOps Server**, **Amazon ECS**, an **Application Load Balancer**, **Amazon ECR**, and **Amazon CloudWatch**.
- The public subnets provide internet connectivity through the **Internet Gateway** and host the internet-facing **Application Load Balancer** and the **NAT Gateway**.
- The **EC2 DevOps Server** is hosted in a public subnet and runs the required DevOps tools.
- The ECS application runs in the two private subnets, with the **NAT Gateway** providing outbound internet access for the private subnets.
- **Amazon ECR** is used as the container image registry.

Configuration Management

- **Ansible** is used to configure the **EC2 DevOps Server**.
- The server is configured with the required DevOps tools, including **Jenkins**, **SonarQube**, and **Grafana**.
- Ansible inventory and playbooks are used to automate the configuration process.

CI/CD Pipeline

- Developers push application code to **AWS CodeCommit**.
- **Jenkins** retrieves the code and automates the CI/CD workflow.
- The pipeline builds the application, runs tests, and creates the Docker image.
- **SonarQube** performs code quality and security analysis.
- **Trivy** scans the Docker image for vulnerabilities.
- A **security gate** evaluates the SonarQube and Trivy results and determines whether the pipeline can continue or stop.
- Validated images are pushed to **Amazon ECR**.

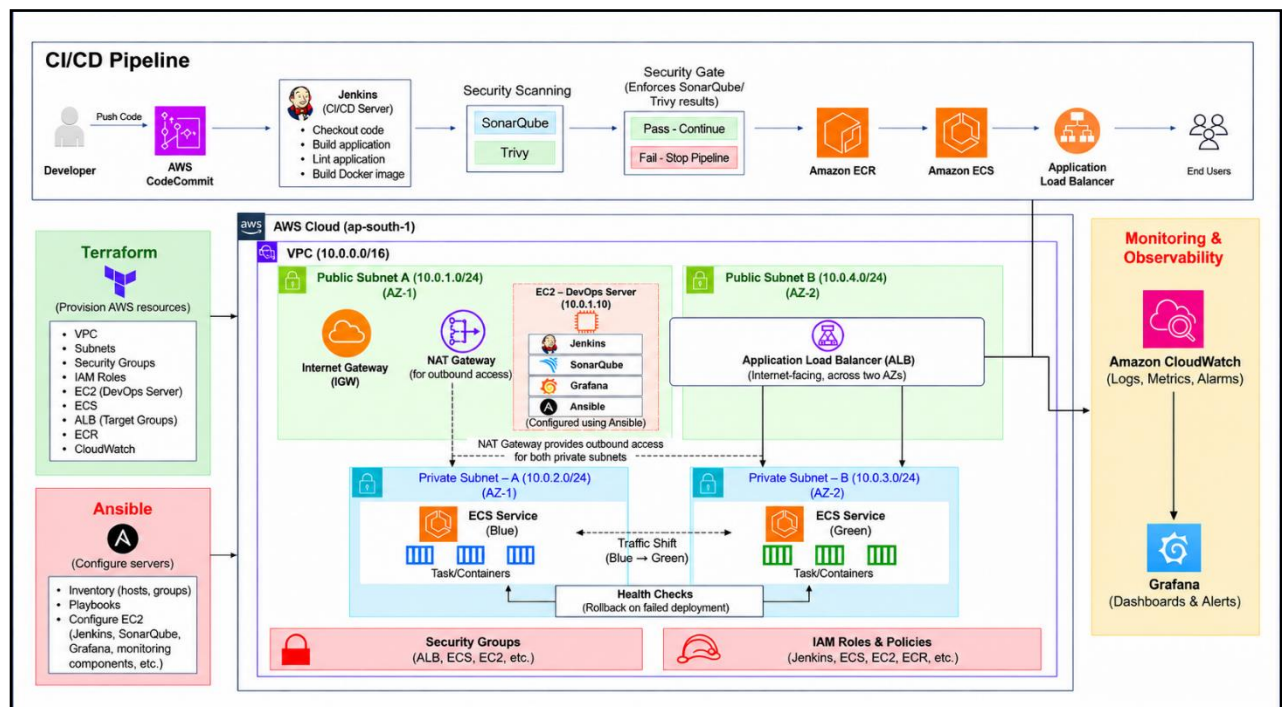
Application Deployment

- **Amazon ECS** runs the containerized application in the private subnets.
- The application uses a **Blue/Green deployment strategy** with separate Blue and Green ECS services.
- **Application Load Balancer** is deployed across the two public subnets and uses separate target groups for the Blue and Green environments.
- Health checks are performed on the ECS services before traffic is shifted to the new environment.
- Traffic can be shifted between the Blue and Green environments through the Application Load Balancer.
- Failed deployments can be rolled back to the previous environment.

Monitoring and Observability

- **Amazon CloudWatch** collects application logs, ECS metrics, and Application Load Balancer metrics.
- **CloudWatch Alarms** are used to monitor important infrastructure and application conditions.
- **Grafana** is configured with **Amazon CloudWatch as the data source** to visualize the collected metrics through monitoring dashboards.
- Grafana dashboards provide visibility into the application and AWS infrastructure performance.

Architecture Diagram



Objectives

The main objectives are:

- Design a production-like AWS infrastructure using **Terraform**.
- Create the required **VPC**, **two public subnets**, **two private subnets**, **security groups**, **IAM roles**, **ECS infrastructure**, and **Application Load Balancer**.
- Use **Ansible** to create inventories, write playbooks, and automate server configuration.
- Build an automated **Jenkins CI/CD pipeline** for application linting, build validation, and Docker image creation.
- Integrate **SonarQube** for static code analysis and **Trivy** for Docker image vulnerability scanning.
- Implement **security gates** to control the progression of the pipeline.
- Store validated Docker images in **Amazon ECR** and deploy them to **Amazon ECS**.
- Implement **Blue/Green deployment** with ALB target groups, application health checks, and rollback.

- Configure **Amazon CloudWatch** logs, metrics, and alarms for ECS services and application health.
- Create **Grafana dashboards** using CloudWatch as the data source for monitoring and visualization.
- Maintain the project code, infrastructure configuration, automation scripts, and documentation in a **Git repository**.

Prerequisites

Before starting the implementation, make sure the following requirements are met:

- An **AWS account** with access to create the required AWS resources.
- **AWS CLI** installed and configured with appropriate credentials.
- **Git** installed for source-code management.
- A **Git repository** for the application code, infrastructure code, Ansible playbooks, Jenkins configuration, and documentation.
- **Terraform** installed for AWS infrastructure provisioning.
- **Ansible** installed for server configuration.
- **Docker** installed for building and testing container images.
- A **Jenkins environment** available on the DevOps EC2 server.
- **SonarQube and Trivy** available for code quality and vulnerability scanning.
- **Grafana** available for monitoring dashboards and configured to use **Amazon CloudWatch** as its data source.
- Basic familiarity with **AWS, Linux, Git, Docker, Terraform, Ansible, Jenkins, and CI/CD concepts**.
- A **React + Vite web application** with its source code and required build configuration.

Step 1: Application Containerization with Docker

The **Quiz App**, developed using React and Vite, is selected as the application. The application source code is available in the project repository below.

[Quiz App](#)

The application is containerized using Docker.

1.1 Create the Dockerfile

Create a Dockerfile in the root directory of the Quiz App to define how the React + Vite application is built and packaged into a Docker image.

Dockerfile

```
FROM node:22-alpine AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

The Dockerfile uses a **multi-stage build**. The first stage installs the application dependencies and generates the production build using Vite. The second stage uses Nginx to serve the generated static files, resulting in a lightweight production container.

1.2 Build and Verify the Docker Image

Build the Docker image using the Dockerfile

```
$ docker build -t quiz-app .
```

```
root@ip-10-0-1-225:/home/ubuntu/Quiz-App# docker build -t quiz-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

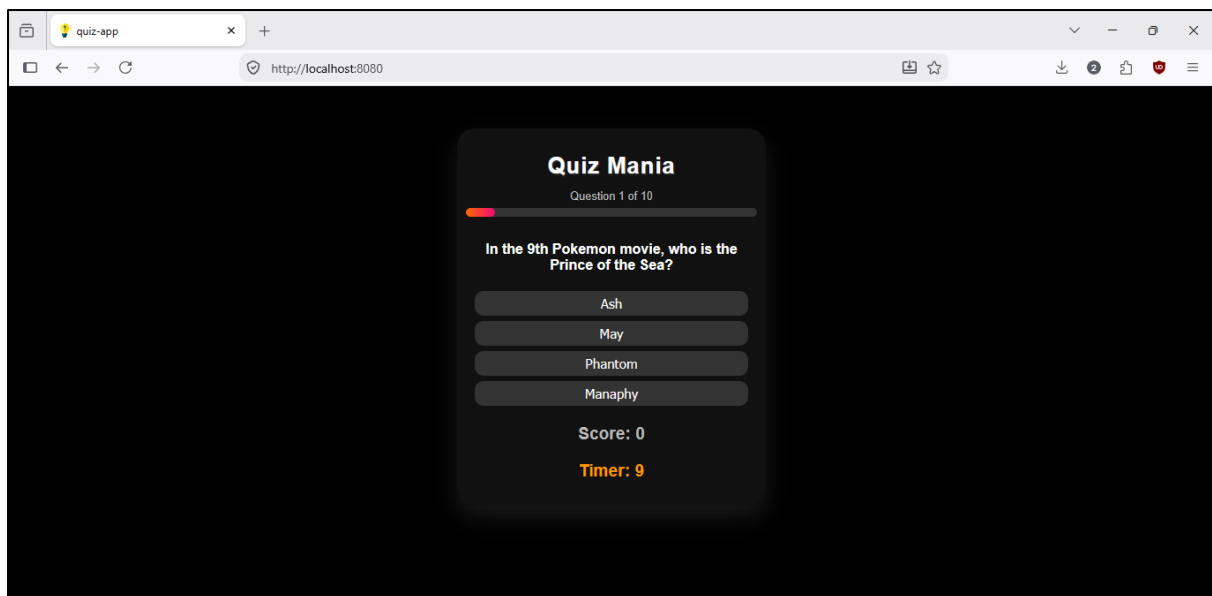
Sending build context to Docker daemon 176.6kB
Step 1/10 : FROM node:22-alpine AS builder
22-alpine: Pulling from library/node
65213f1a3de8: Pulling fs layer
1a3a899a8294: Pulling fs layer
e2de96513ba9: Pulling fs layer
```

Run the container locally to verify that the application is served correctly.

```
$ docker run -d -p 8080:80 --name quiz-app quiz-app
```

Open the application in a browser.

```
http://localhost:8080
```



Step 2: Infrastructure Provisioning with Terraform

2.1 Create the Terraform Project

Create the Terraform project and organize the configuration into separate .tf files based on the resources being provisioned.

The project structure is as follows:-

```
terraform/  
├── provider.tf  
├── variables.tf  
├── vpc.tf  
├── subnets.tf  
├── route_tables.tf  
├── security_groups.tf  
├── iam.tf  
├── ec2.tf  
├── ecr.tf  
├── ecs.tf  
├── alb.tf  
└── outputs.tf
```

The AWS provider was configured to define the AWS region in which the infrastructure would be deployed.

variables.tf

```
variable "aws_region" {  
    description = "AWS region for the infrastructure"  
    type       = string  
    default    = "ap-south-1"  
}
```

provider.tf

```
provider "aws" {  
    region = var.aws_region  
}
```

Initialize the Terraform configuration.

```
$ terraform init
```

```
Terraform has been successfully initialized!
```

```
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.
```

2.2 Configure the VPC and Network

vpc.tf

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "capstone-vpc"
  }
}
```

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Bl
☐	capstone-vpc	vpc-0eb1c7a735d480393	✓ Available	-	-	⊞

Divide the VPC into **two public subnets across separate Availability Zones** and **two private subnets across separate Availability Zones**.

The public subnets host the internet-facing Application Load Balancer, DevOps EC2 server, and NAT Gateway. The private subnets host the ECS application workloads.

subnets.tf

```
resource "aws_subnet" "public_a" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = "10.0.1.0/24"
  availability_zone = "<AZ_1>"

  tags = {
    Name = "public-subnet-a"
  }
}
```

```
resource "aws_subnet" "public_b" {
  vpc_id      = aws_vpc.main.id
  cidr_block   = "10.0.4.0/24"
  availability_zone = "<AZ_2>"

  tags = {
    Name = "public-subnet-b"
  }
}

resource "aws_subnet" "private_a" {
  vpc_id      = aws_vpc.main.id
  cidr_block   = "10.0.2.0/24"
  availability_zone = "<AZ_1>"

  tags = {
    Name = "private-subnet-a"
  }
}

resource "aws_subnet" "private_b" {
  vpc_id      = aws_vpc.main.id
  cidr_block   = "10.0.3.0/24"
  availability_zone = "<AZ_2>"

  tags = {
    Name = "private-subnet-b"
  }
}
```

Replace <AZ_1> and <AZ_2> with two Availability Zones in the selected AWS region.

☐	public-subnet-b	subnet-0a44712c0732ece34	✔ Available	vpc-0eb1c7a735d480393 caps...	⊖ Off
☐	private-subnet-b	subnet-0c6e9761d59b34733	✔ Available	vpc-0eb1c7a735d480393 caps...	⊖ Off
☐	public-subnet-a	subnet-04575b40f8509f333	✔ Available	vpc-0eb1c7a735d480393 caps...	⊖ Off
☐	private-subnet-a	subnet-0cf18876f6279016f	✔ Available	vpc-0eb1c7a735d480393 caps...	⊖ Off

Internet Gateway

Create and attach an Internet Gateway to the VPC.

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "capstone-igw"
  }
}
```

☐	Name	Internet gateway ID	State	VPC ID
☐	capstone-igw	igw-0d785d0aaa4d5e6d9	✔ Attached	vpc-0eb1c7a735d480393 capstone-vpc

NAT Gateway

Create an Elastic IP and NAT Gateway in **Public Subnet A** to provide outbound internet access to resources in the private subnets.

```
resource "aws_eip" "nat" {
  domain = "vpc"
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public_a.id

  tags = {
    Name = "capstone-nat"
  }
}
```

	Name ▾	NAT gateway ID ▾	Connectivity... ▾	State ▾	State message ▾	Availability ... ▾
	capstone-nat	nat-01344e9145bdc42da	Public	 Available	-	Zonal

Public Route Table

Create a public route table with internet access through the Internet Gateway.

```
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "public-route-table"
  }
}
```

Private Route Table

Create a private route table that uses the NAT Gateway for outbound internet access.

```
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }

  tags = {
    Name = "private-route-table"
  }
}
```

☐	private-route-table	rtb-033a45d0fc65d3767	2 subnets	-	No	vpc-0
☐	public-route-table	rtb-0e3d0f943ca1dbac7	2 subnets	-	No	vpc-0

Associate the Route Tables

Associate the public route table with both public subnets and the private route table with both private subnets.

```
resource "aws_route_table_association" "public_a" {
  subnet_id    = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_b" {
  subnet_id    = aws_subnet.public_b.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private_a" {
  subnet_id    = aws_subnet.private_a.id
  route_table_id = aws_route_table.private.id
}

resource "aws_route_table_association" "private_b" {
  subnet_id    = aws_subnet.private_b.id
  route_table_id = aws_route_table.private.id
}
```

2.3 Configure Security Groups

Create separate security groups for the **Application Load Balancer**, **ECS workloads**, and **DevOps EC2 server**.

ALB Security Group

Allow HTTP traffic from the internet to the Application Load Balancer.

security_groups.tf

```
resource "aws_security_group" "alb" {
  name = "alb-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "alb-sg"
  }
}
```

ECS Security Group

Allow application traffic from the ALB security group to the ECS tasks.

security_groups.tf

```
resource "aws_security_group" "ecs" {
  name = "ecs-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"

    security_groups = [aws_security_group.alb.id]
  }
}
```

```

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
tags = {
  Name = "ecs-sg"
}
}

```

DevOps EC2 Security Group

Allow SSH access to the DevOps server. Restrict SSH to the administrator's IP address instead of allowing access from the entire internet.

security_groups.tf

```

resource "aws_security_group" "devops" {
  name = "devops-sg"
  vpc_id = aws_vpc.main.id
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["<YOUR_IP>/32"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    Name = "devops-sg"
  }
}

```

Replace <YOUR_IP> with the public IP address from which the DevOps server will be administered.

☐	alb-sg	sg-04261ec17b14d3cd4	alb-sg	vpc-0eb1c7a735d480393
☐	ecs-sg	sg-07af5a534f264c6b9	ecs-sg	vpc-0eb1c7a735d480393
☐	devops-sg	sg-077ecc59eb80836fb	devops-sg	vpc-0eb1c7a735d480393

2.4 Configure IAM Roles and Policies

Create the IAM roles required for ECS and the DevOps EC2 server.

ECS Task Execution Role

Create the role that allows ECS to perform tasks such as pulling container images from ECR and sending logs to CloudWatch.

iam.tf

```
resource "aws_iam_role" "ecs_task_execution_role" {
  name = "capstone-ecs-task-execution-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect = "Allow"

        Principal = {
          Service = "ecs-tasks.amazonaws.com"
        }

        Action = "sts:AssumeRole"
      }
    ]
  })
}
```

```
resource "aws_iam_role_policy_attachment"
"ecs_task_execution_policy" {

  role = aws_iam_role.ecs_task_execution_role.name

  policy_arn = "arn:aws:iam::aws:policy/service-
role/AmazonECSTaskExecutionRolePolicy"
}
```

ECS Task Role

Create a task role for permissions required by the application running inside the ECS container.

iam.tf

```
resource "aws_iam_role" "ecs_task_role" {
  name = "capstone-ecs-task-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect = "Allow"

        Principal = {
          Service = "ecs-tasks.amazonaws.com"
        }

        Action = "sts:AssumeRole"
      }
    ]
  })
}
```

DevOps EC2 Server Role

Create an IAM role for the DevOps EC2 server so that Jenkins can interact with the AWS resources required by the pipeline.

iam.tf

```
resource "aws_iam_role" "devops_ec2_role" {
  name = "capstone-devops-ec2-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect = "Allow"

        Principal = {
          Service = "ec2.amazonaws.com"
        }

        Action = "sts:AssumeRole"
      }
    ]
  })
}
```

Attach the required ECR and ECS permissions.

```
resource "aws_iam_role_policy_attachment" "devops_ecr_policy" {
  role = aws_iam_role.devops_ec2_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryPowerUser"
}
```

```

resource "aws_iam_role_policy_attachment" "devops_ecs_policy" {
  role = aws_iam_role.devops_ec2_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonECS_FullAccess"
}
resource "aws_iam_role_policy_attachment" "devops_cloudwatch" {
  role = aws_iam_role.devops_ec2_role.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchReadOnlyAccess"
}

```

Create an instance profile so the IAM role can be attached to the EC2 server.

```

resource "aws_iam_instance_profile" "devops_ec2_profile" {
  name = "capstone-devops-ec2-profile"
  role = aws_iam_role.devops_ec2_role.name
}

```

The ECS task execution role provides access to ECR and CloudWatch Logs, while the DevOps EC2 role provides Jenkins with access to the AWS resources required by the pipeline.

2.5 Provision the DevOps EC2 Server

Create an EC2 instance in **Public Subnet A** to host the DevOps tools, including Jenkins, SonarQube, and Grafana.

ec2.tf

```

resource "aws_instance" "devops_server" {
  ami      = "ami-xxxxxxxxxxxxxxxxxx"
  instance_type = "t3.small"
  subnet_id = aws_subnet.public_a.id
  vpc_security_group_ids = [
    aws_security_group.devops.id
  ]
  iam_instance_profile = aws_iam_instance_profile.devops_ec2_profile.name

  key_name = "capstone-key"
}

```

```

tags = {
  Name = "capstone-devops-server"
}
}

resource "aws_eip" "devops_server" {
  domain = "vpc"

  tags = {
    Name = "capstone-devops-eip"
  }
}

resource "aws_eip_association" "devops_server" {
  instance_id = aws_instance.devops_server.id
  allocation_id = aws_eip.devops_server.id
}

```



Replace the AMI ID with the Ubuntu AMI ID available in the AWS region selected in provider.tf.

The EC2 instance uses **Public Subnet A**, the DevOps security group, the IAM instance profile, and the capstone-key key pair.

Create an EC2 key pair named capstone-key in AWS and specify the same name in key_name.

2.6 Configure Amazon ECR

Create an Amazon ECR repository to store the Docker images generated by the Jenkins pipeline.

ecr.tf

```

resource "aws_ecr_repository" "app" {
  name          = "capstone-app"
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
}

```

```
tags = {
  Name = "capstone-app"
}
```

	Repository name ▲	URI	Created at ▼	Tag immutability	Encryption type
☐	capstone-app	 653455484229.dk r.ecr.ap-south-1.amazonaws.com/capstone-app	September 22, 2026, 23:23:22 (UTC+05.5)	Mutable	AES-256

2.7 Configure ECS and Application Load Balancer

Create the ECS cluster required to run the containerized application.

ecs.tf

```
resource "aws_ecs_cluster" "main" {
  name = "capstone-cluster"
}
```

Cluster ▼	Services ▼	Tasks ▼	Container instances ▼
capstone-cluster	0	No tasks running	0 EC2

Create the Application Load Balancer across the two public subnets.

alb.tf

```
resource "aws_lb" "app" {
  name           = "capstone-alb"
  load_balancer_type = "application"

  security_groups = [
    aws_security_group.alb.id
  ]

  subnets = [
    aws_subnet.public_a.id,
    aws_subnet.public_b.id
  ]
}
```

```
tags = {  
  Name = "capstone-alb"  
}  
}
```

☐	Name	State	Type	Scheme	IP address type
☐	capstone-alb	✓ Active	application	Internet-facing	IPv4

The Application Load Balancer receives application traffic and provides the entry point for the Blue/Green ECS deployment.

The two public subnets allow the internet-facing ALB to operate across two Availability Zones.

2.8 Apply and Verify the Infrastructure

Validate the Configuration

```
$ terraform validate
```

```
D:\AWS\Capstone Project\Quiz-App\Terraform>terraform validate  
Success! The configuration is valid.
```

Review the Execution Plan.

```
$ terraform plan
```

```
D:\AWS\Capstone Project\Quiz-App\Terraform>terraform plan  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:  
+ create  
  
Terraform will perform the following actions:  
  
# aws_ecr_repository.app will be created  
+ resource "aws_ecr_repository" "app" {  
  + arn              = (known after apply)  
  + id              = (known after apply)  
  + image_tag_mutability = "MUTABLE"  
  + name            = "capstone-app"  
}
```

Apply the Configuration

```
$ terraform apply
```

```
D:\AWS\Capstone Project\Quiz-App\Terraform>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_ecr_repository.app will be created
+ resource "aws_ecr_repository" "app" {
+   arn                = (known after apply)
+   id                 = (known after apply)
+   image_tag_mutability = "MUTABLE"
+   name               = "capstone-app"
+   region             = "ap-south-1"
}
```

☐	Name 	Instance ID	Instance state 	Instance type 	Status check	Availability Zone 
☐	capstone-devo...	i-0ca31ffb875cbb19f	 Running 	t3.small	 3/3 checks passed	ap-south-1a

Step 3: Server Configuration with Ansible

3.1 Configure the Ansible Inventory

Create an Ansible inventory containing the public IP address of the DevOps EC2 server created by Terraform.

inventory.ini

```
[devops_server]

devops-server ansible_host=<EC2_PUBLIC_IP> ansible_user=ubuntu
```

Replace <EC2_PUBLIC_IP> with the public IP address of the DevOps EC2 instance.

Verify connectivity before running the playbook.

```
$ ansible -i inventory.ini devops_server -m ping
```

```
devops-server | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.14"
  },
  "changed": false,
  "ping": "pong"
}
```

3.2 Create the Ansible Playbook

Create a playbook to configure the DevOps EC2 server with the required packages and tools.

devops_setup.yml

```
- name: Configure DevOps Server
  hosts: devops_server
  become: true

  tasks:

    - name: Update package cache
      apt:
        update_cache: yes

    - name: Install required system packages
      apt:
        name:
          - git
          - docker.io
          - unzip
          - curl
          - wget
          - fontconfig
          - openjdk-21-jre
        state: present

    - name: Start Docker service
      service:
        name: docker
        state: started
        enabled: true

    - name: Add Ubuntu user to Docker group
      user:
        name: ubuntu
        groups: docker
        append: true
```

- name: Install Node.js and npm

shell: |

```
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -  
apt-get install -y nodejs
```

args:

executable: /bin/bash

- name: Install AWS CLI

shell: |

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-  
x86_64.zip" \
```

```
-o "/tmp/awsclicv2.zip"
```

```
rm -rf /tmp/aws
```

```
unzip -q /tmp/awsclicv2.zip -d /tmp
```

```
/tmp/aws/install --update
```

args:

executable: /bin/bash

- name: Install Trivy repository

shell: |

```
wget -qO - https://aquasecurity.github.io/trivy-  
repo/deb/public.key \
```

```
| gpg --dearmor \
```

```
| tee /usr/share/keyrings/trivy.gpg > /dev/null
```

```
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
```

```
https://aquasecurity.github.io/trivy-repo/deb generic  
main" \
```

```
| tee /etc/apt/sources.list.d/trivy.list
```

```
apt-get update
```

args:

executable: /bin/bash

- name: Install Trivy

- name: Install Trivy

apt:

name: trivy

state: present

- name: Add Jenkins repository key

shell: |

mkdir -p /etc/apt/keyrings

wget -O /etc/apt/keyrings/jenkins-keyring.asc \

https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

args:

executable: /bin/bash

- name: Add Jenkins repository

shell: |

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
\

https://pkg.jenkins.io/debian-stable binary/" \

| tee /etc/apt/sources.list.d/jenkins.list

apt-get update

args:

executable: /bin/bash

- name: Install Jenkins

apt:

name: jenkins

state: present

- name: Add Jenkins user to Docker group

user:

name: jenkins

groups: docker

append: true

- name: Start Jenkins

service:

name: jenkins

state: started

enabled: true

- name: Pull SonarQube image

community.docker.docker_image:

name: sonarqube:community

source: pull

- name: Run SonarQube

community.docker.docker_container:

name: sonarqube

image: sonarqube:community

state: started

restart_policy: unless-stopped

ports:

- "9000:9000"

- name: Pull Grafana image

community.docker.docker_image:

name: grafana/grafana

source: pull

- name: Run Grafana

community.docker.docker_container:

name: grafana

image: grafana/grafana

state: started

restart_policy: unless-stopped

ports:

- "3000:3000"

Ansible Docker Collection

The playbook uses the community.docker collection for the SonarQube and Grafana containers.

Install the collection on the machine from which Ansible is executed.

```
$ ansible-galaxy collection install community.docker
```

```
# /usr/lib/python3/dist-packages/ansible/_internal/ansible_collections
Collection                                     Version
-----
ansible._protomatter                         *
```

```
# /usr/lib/python3/dist-packages/ansible_collections
Collection                                     Version
-----
amazon.aws                                   10.1.2
ansible.netcommon                           8.2.0
```

3.3 Execute the Ansible Playbook

Run the playbook from the machine where Ansible is installed.

```
$ ansible-playbook -i inventory.ini devops_setup.yml
```

```
root@ip-10-0-1-225:~/ansible# ansible-playbook -i inventory.ini devops_setup.yml

PLAY [Configure DevOps Server] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'devops-server' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [devops-server]

TASK [Update package cache] *****
changed: [devops-server]

TASK [Install required system packages] *****
ok: [devops-server]

TASK [Start Docker service] *****
ok: [devops-server]
```

3.4 Configure Jenkins Access to Docker

Jenkins needs permission to communicate with the Docker daemon because the Jenkins pipeline builds Docker images.

Start Jenkins.

```
$ sudo systemctl restart jenkins
```

Verify the status.

```
$ sudo systemctl status jenkins
```

```
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-09-23 03:12:12 UTC; 16s ago
```

3.5 Configure the DevOps Tools

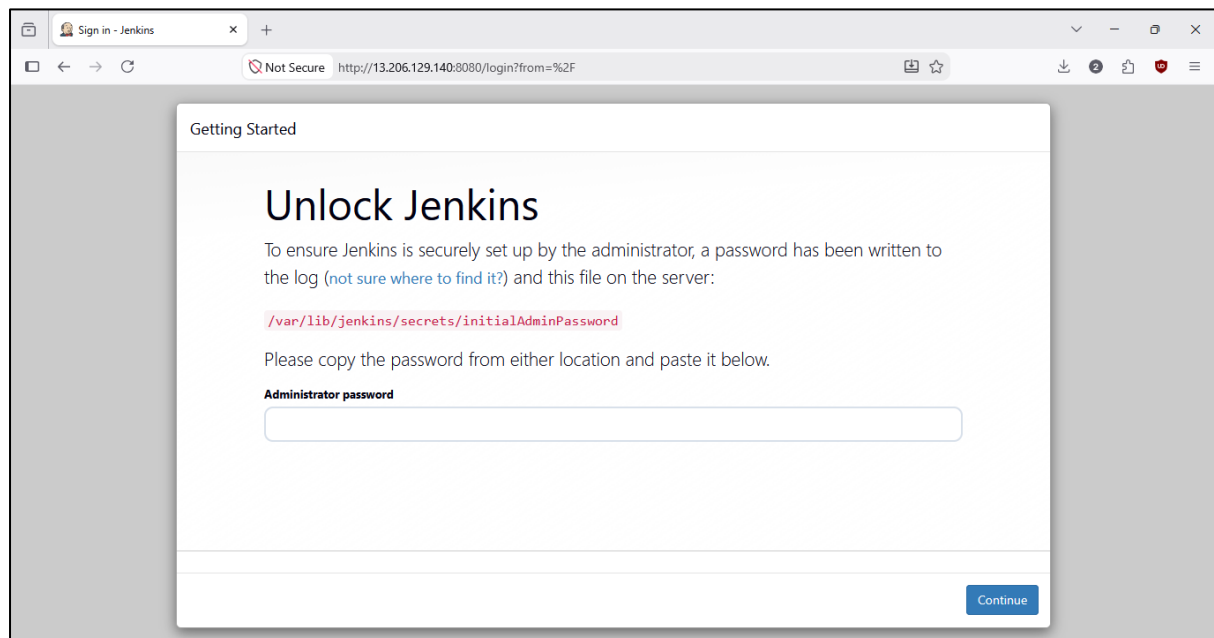
Configure the DevOps EC2 security group to allow access to the required DevOps tools.

- Jenkins (Port 8080)
- SonarQube (Port 9000)
- Grafana (Port 3000)

Jenkins

After Ansible completes, access the tools using the EC2 public IP.

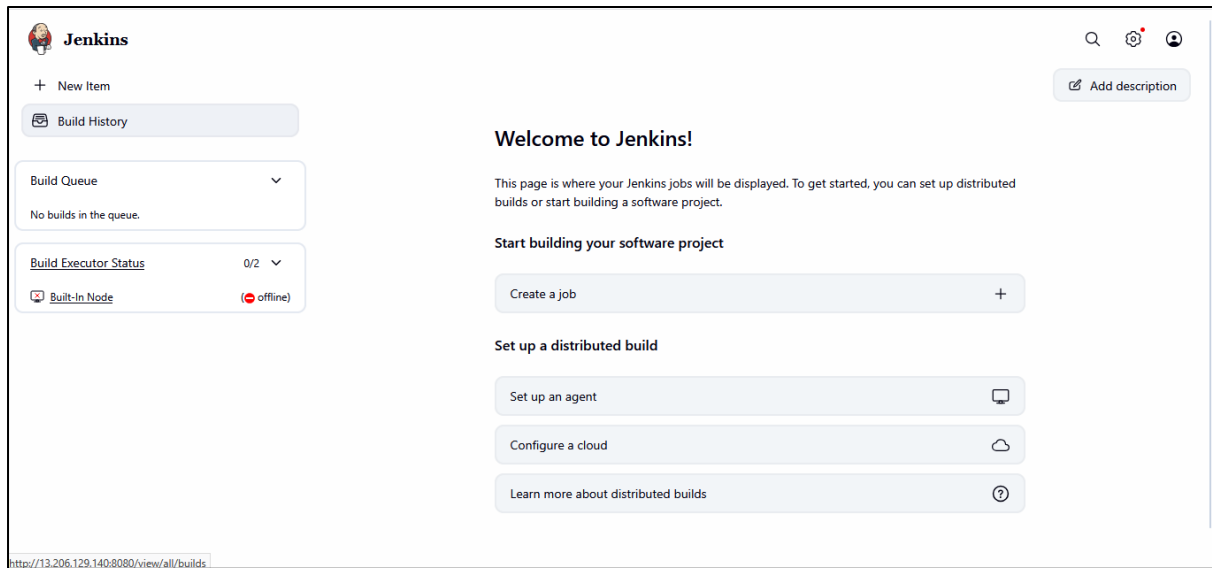
```
http://<EC2_PUBLIC_IP>:8080
```



Retrieve the initial Jenkins administrator password.

```
$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

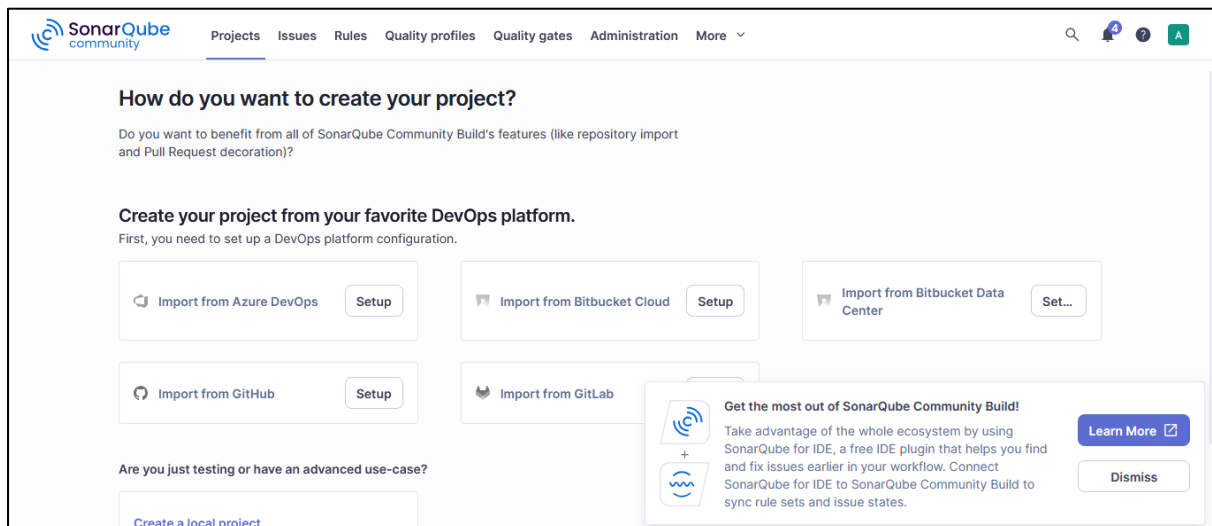
Use the Jenkins setup wizard to install the required plugins and create the administrator account. Jenkins' official installation process uses this initial password during the first setup.



SonarQube

Verify that the SonarQube interface is accessible.

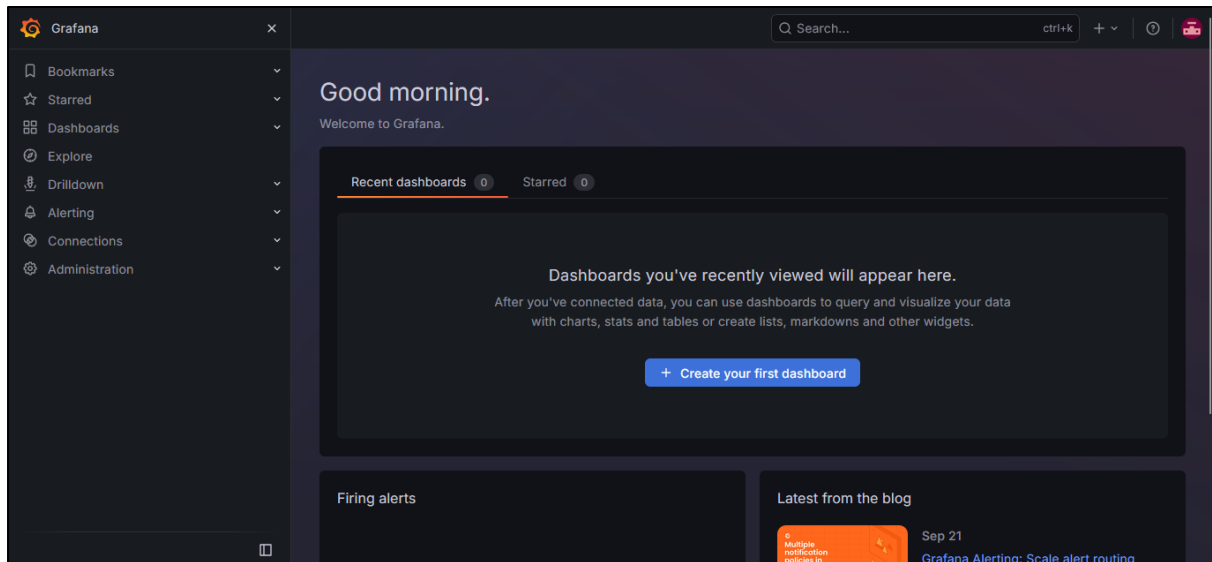
`http://<EC2_PUBLIC_IP>:9000`



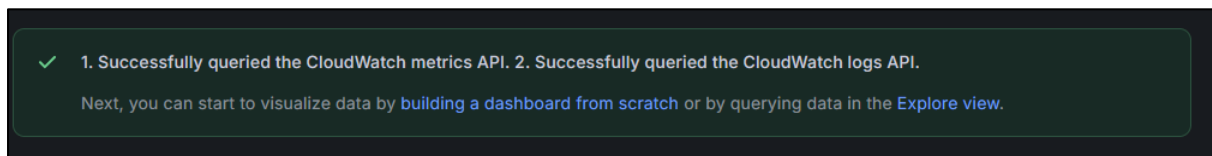
Grafana

Verify that Grafana is accessible.

`http://<EC2_PUBLIC_IP>:3000`



Configure Amazon CloudWatch as the Grafana data source.



3.6 Configure SonarQube Scanner

The Jenkins pipeline performs code analysis using the SonarQube scanner.

Install the SonarQube Scanner CLI on the DevOps server and make it available to Jenkins.

```
root@ip-10-0-1-92:/tmp# sonar-scanner --version
06:24:00.787 INFO Scanner configuration file: /opt/sonar-scanner-8.1.0.6389-linux-x64/conf/sonar-scanner.properties
06:24:00.800 INFO Project root configuration file: NONE
06:24:00.837 INFO SonarScanner CLI 8.1.0.6389
06:24:00.858 INFO Linux 7.0.0-1006-aws amd64
```

Step 4: Jenkins CI/CD Pipeline

The Jenkins pipeline automates the application delivery workflow. It retrieves the application source code from AWS CodeCommit, installs dependencies, validates and builds the React + Vite application, performs SonarQube code analysis, builds and scans the Docker image using Trivy, and pushes the validated image to Amazon ECR.

4.1 Configure the CodeCommit Repository

- Create an AWS CodeCommit repository to serve as the source repository for the Jenkins pipeline.

	Name ▾	Description	Last modified ▾	Clone URL
☐	quiz-app	React + Vite Quiz App for AWS DevOps Capstone	Just now	HTTPS SSH HTTPS (GRC)

- Transfer the React + Vite Quiz App source code from the GitHub repository to the CodeCommit repository. Jenkins uses the CodeCommit repository as the source for the CI/CD pipeline.
- Verify that the application source code, package.json, package-lock.json, Dockerfile, and other required project files are available in the CodeCommit repository.

quiz-app

Reference

[Notify](#)

master ▾

[Create pull request](#)
[Clone URL ▾](#)

quiz-app

Add file ▾

	Name
📁	public
📁	src
📄	Dockerfile
📄	eslint.config.js
📄	index.html
📄	package-lock.json
📄	package.json

4.2 Configure Jenkins to Access CodeCommit

Jenkins retrieves the application source code from CodeCommit when the pipeline is triggered.

Verify the AWS CLI configuration on the DevOps server.

```
$ aws sts get-caller-identity
```

Verify CodeCommit access.

```
$ aws codecommit list-repositories \
--region <AWS_REGION>
```

```
root@ip-10-0-1-92:/home/ubuntu# aws codecommit list-repositories --region ap-south-1
{
  "repositories": [
    {
      "repositoryName": "quiz-app",
      "repositoryId": "a24491f7-d20d-458e-9529-1a7600cfe657"
    }
  ]
}
```

4.3 Create the Jenkins Pipeline

Create a Jenkinsfile in the root directory of the application repository.

Jenkinsfile

```
pipeline {
  agent any

  environment {
    AWS_REGION = '<AWS_REGION>'
    AWS_ACCOUNT_ID = '<AWS_ACCOUNT_ID>'
    ECR_REPOSITORY = 'capstone-app'
    IMAGE_TAG = "${BUILD_NUMBER}"
    ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'master',
          url: '<CODECOMMIT_REPOSITORY_URL>'
      }
    }

    stage('Install Dependencies') {
      steps {
        sh 'npm ci'
      }
    }

    stage('Lint') {
      steps {
        sh 'npm run lint'
      }
    }
  }
}
```

```

}

stage('Build Application') {
    steps {
        sh 'npm run build'
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh """
            sonar-scanner \
            -Dsonar.projectKey=quiz-app \
            -Dsonar.sources=src
            """
        }
    }
}

stage('Build Docker Image') {
    steps {
        sh "docker build -t ${ECR_REPOSITORY}:${IMAGE_TAG} ."
    }
}

stage('Trivy Scan') {
    steps {
        sh """
        trivy image \
        --exit-code 1 \
        --severity HIGH,CRITICAL \
        ${ECR_REPOSITORY}:${IMAGE_TAG}
        """
    }
}

```

```

}

stage('Security Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}

stage('Login to ECR') {
    steps {
        sh '''
            aws ecr get-login-password --region ${AWS_REGION} |
            docker login --username AWS --password-stdin ${ECR_REGISTRY}
        '''
    }
}

stage('Push to ECR') {
    steps {
        sh '''
            docker tag ${ECR_REPOSITORY}:${IMAGE_TAG} \
            ${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}

            docker push \
            ${ECR_REGISTRY}/${ECR_REPOSITORY}:${IMAGE_TAG}
        '''
    }
}
}
}

```

The pipeline uses `${BUILD_NUMBER}` as the Docker image tag so that each Jenkins build produces a uniquely identifiable image.

4.4 Build and Validate the Application

The pipeline installs the application dependencies and performs the required validation before creating the Docker image.

```
$ npm ci
$ npm run lint
$ npm run build
```

The `npm ci` command installs the dependencies defined in `package-lock.json`. The lint stage checks the source code for errors and the build stage generates the production-ready Vite build.

4.5 Integrate SonarQube Code Analysis

SonarQube is integrated into Jenkins to analyze the application source code for code quality and security issues.

Configure the SonarQube server in Jenkins and the SonarQube Scanner required by Jenkins.

The pipeline then executes.

```
stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
            sonar-scanner \
            -Dsonar.projectKey=quiz-app \
            -Dsonar.sources=src
            '''
        }
    }
}
```

The analysis result is sent to the SonarQube server for evaluation.

4.6 Build the Docker Image

After the application passes the dependency installation, linting, and build stages, Jenkins builds the Docker image using the **Dockerfile**.

```
$ docker build -t capstone-app:${BUILD_NUMBER} .
```

The image is tagged using the Jenkins build number.

Verify the generated image.

```
$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
653455484229.dkr.ecr.ap-south-1.amazonaws.com/capstone-app:6	c152ae8f0055	93.9MB	26.4MB	
capstone-app:5	c152ae8f0055	93.9MB	26.4MB	
capstone-app:6	c152ae8f0055	93.9MB	26.4MB	
grafana/grafana:latest	ac461fb352ab	1.92GB	476MB	U
nginx:alpine	1ed1b0e1d765	94.4MB	27.2MB	
node:22-alpine	b6f26b36c8ff	238MB	61.2MB	
sonarqube:community	58b068af30bd	2.54GB	1.09GB	U

4.7 Scan the Docker Image with Trivy

Trivy scans the generated Docker image for known vulnerabilities.

```
trivy image \  
  --exit-code 1 \  
  --severity HIGH,CRITICAL \  
  capstone-app:${BUILD_NUMBER}
```

The configured policy causes the pipeline to stop when HIGH or CRITICAL vulnerabilities are detected.

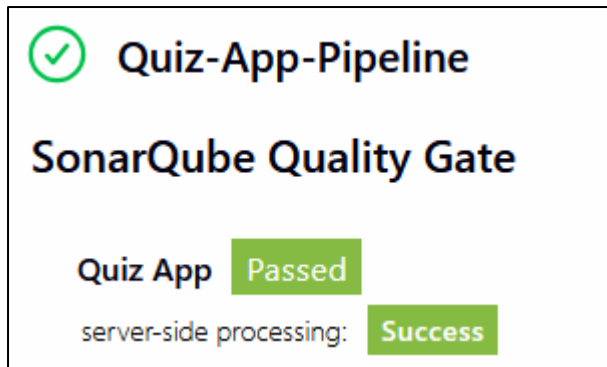
If no vulnerabilities meeting the configured severity threshold are detected, the pipeline continues to the security gate.

4.8 Configure the Security Gate

The security gate ensures that the application proceeds to Amazon ECR only after the required security checks have passed.

The Jenkins pipeline waits for the SonarQube quality gate result.

```
stage('Security Gate') {  
  steps {  
    timeout(time: 5, unit: 'MINUTES') {  
      waitForQualityGate abortPipeline: true  
    }  
  }  
}
```



If the SonarQube quality gate fails, Jenkins stops the pipeline.

A HIGH or CRITICAL vulnerability causes the Trivy command to return a failure status, which stops the Jenkins pipeline.

4.9 Push the Validated Image to Amazon ECR

After the application passes the required validation and security checks, Jenkins authenticates with Amazon ECR.

```
aws ecr get-login-password \  
  --region <AWS_REGION> \  
docker login \  
  --username AWS \  
  --password-stdin \  
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com
```

Tag the Docker image with the ECR repository URI.

```
docker tag capstone-app:${BUILD_NUMBER} \  
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com/capstone-app:${BUILD_NUMBER}
```

Push the image to Amazon ECR.

```
docker push \  
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com/capstone-app:${BUILD_NUMBER}
```

Verify that the image is available in the ECR repository.

```
aws ecr describe-images \  
  --repository-name capstone-app \  
  --region <AWS_REGION>
```

☐	Image tags ▾	Type	Created at ▾	Image size (MB) ▾	Image digest	Last pulled at ▾
☐	6	Image	September 23, 2026, 15:52:11 (UTC+05.5)	26.42	sha256:c15...	-

The validated Docker image is now stored in **Amazon ECR** and is ready to be deployed to **Amazon ECS**.

Step 5: Application Deployment to Amazon ECS

The validated Docker image stored in Amazon ECR is deployed to Amazon ECS using a Blue/Green deployment approach. Separate Blue and Green ECS services are connected to separate Application Load Balancer target groups. The Green environment is deployed and validated before traffic is shifted from Blue to Green.

5.1 Create the ECS Task Definition

Create an ECS task definition to define how the Quiz App container runs on AWS Fargate. The task definition specifies the ECR image, CPU and memory allocation, container port, and CloudWatch logging configuration.

ecs-task-definition.json

```
{
  "family": "quiz-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "<ECS_TASK_EXECUTION_ROLE_ARN>",
  "containerDefinitions": [
    {
      "name": "quiz-app",
      "image":
"<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com/cap
stone-app:<IMAGE_TAG>",
      "essential": true,
      "portMappings": [
        {
```

```

    "containerPort": 80,
    "protocol": "tcp"
  },
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/quiz-app",
      "awslogs-region": "<AWS_REGION>",
      "awslogs-stream-prefix": "ecs"
    }
  }
}
]
}

```

Replace:

- <AWS_ACCOUNT_ID> with the AWS account ID.
- <AWS_REGION> with the AWS region.
- <IMAGE_TAG> with the validated image tag generated by Jenkins.
- <ECS_TASK_EXECUTION_ROLE_ARN> with the ECS task execution role ARN.

Using the Jenkins build number ensures that ECS deploys the exact Docker image that passed the CI/CD pipeline.

The task uses **AWS Fargate**, so the underlying container infrastructure does not need to be managed. The container listens on port 80, which is connected to the Application Load Balancer through the ECS target group.

Register the task definition.

```

aws ecs register-task-definition \
  --cli-input-json file://ecs-task-definition.json \
  --region <AWS_REGION>

```

```

    "family": "quiz-app",
    "executionRoleArn": "arn:aws:iam::xxxxxxxxxxxx:role/capstone-ecs-task-execution-role",
    "networkMode": "awsvpc",
    "revision": 1,
    "volumes": [],
    "status": "ACTIVE",
    "requiresAttributes": [

```

5.2 Configure Blue and Green ECS Services

Create two ECS services for the application.

The **Blue service** represents the existing application environment, while the **Green service** is used to deploy and validate the new application version before traffic is shifted.

Blue Service

```

ECS Cluster: capstone-cluster
Service: quiz-app-blue
Target Group: quiz-app-blue-tg
Desired Tasks: 1

```

Green Service

```

ECS Cluster: capstone-cluster
Service: quiz-app-green
Target Group: quiz-app-green-tg
Desired Tasks: 1

```

```

--query 'services[].[serviceName,status,runningCount,desiredCount,pendingCount]' \
--output table

```

DescribeServices					
serviceName	status	runningCount	desiredCount	pendingCount	
quiz-app-blue	ACTIVE	0	1	0	
quiz-app-green	ACTIVE	0	1	0	

Create the Blue service.

```
aws ecs create-service \  
  --cluster capstone-cluster \  
  --service-name quiz-app-blue \  
  --task-definition quiz-app \  
  --desired-count 1 \  
  --launch-type FARGATE \  
  --network-configuration  
  "awsvpcConfiguration={subnets=[<PRIVATE_SUBNET_A>,<PRIVATE_SUBNET_B>],securityGroups=[<ECS_SECURITY_GROUP_ID>],assignPublicIp=DISABLED}" \  
  --load-balancers "targetGroupArn=<BLUE_TARGET_GROUP_ARN>,containerName=quiz-app,containerPort=80" \  
  --region <AWS_REGION>
```

Create the Green service.

```
aws ecs create-service \  
  --cluster capstone-cluster \  
  --service-name quiz-app-green \  
  --task-definition quiz-app \  
  --desired-count 1 \  
  --launch-type FARGATE \  
  --network-configuration  
  "awsvpcConfiguration={subnets=[<PRIVATE_SUBNET_A>,<PRIVATE_SUBNET_B>],securityGroups=[<ECS_SECURITY_GROUP_ID>],assignPublicIp=DISABLED}" \  
  --load-balancers "targetGroupArn=<GREEN_TARGET_GROUP_ARN>,containerName=quiz-app,containerPort=80" \  
  --region <AWS_REGION>
```

The ECS services run in the private subnets and receive application traffic through the Application Load Balancer.

5.3 Configure Application Load Balancer Target Groups

Create separate target groups for the Blue and Green ECS services.

Blue Service

Name: quiz-app-blue-tg

Target Type: IP

Protocol: HTTP

Port: 80

Health Check Path: /

Green Service

Name: quiz-app-green-tg

Target Type: IP

Protocol: HTTP

Port: 80

Health Check Path: /

DescribeTargetGroups					
quiz-app-blue-tg	arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-blue-tg/9763eaade93ff246	ip	80	/	
quiz-app-green-tg	arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-green-tg/7610a9276362e3dd	ip	80	/	

The target type is set to **IP** because the ECS tasks use the awsvpc network mode.

The Application Load Balancer uses these target groups to route traffic to the corresponding ECS services.

5.4 Configure Health Checks

Configure health checks for both target groups to verify that the Quiz App containers are healthy and able to receive traffic.

Use the application root path for the health check.

Protocol: HTTP

Port: Traffic Port

Health Check Path: /

Healthy Threshold: 2

Unhealthy Threshold: 2

Timeout: 5 seconds

Interval: 30 seconds

The Application Load Balancer periodically checks the application through the configured health-check path.

```
root@ip-10-0-1-225:/home/ubuntu# aws elbv2 describe-target-health \
--target-group-arn arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-blue-tg/9763eaade93ff246 \
--region ap-south-1 \
--query 'TargetHealthDescriptions[][Target.Id,Target.Port,TargetHealth.State,TargetHealth.Reason]' \
--output table
```

DescribeTargetHealth			
10.0.2.49	80	healthy	None
10.0.3.209	80	draining	Target.DeregistrationInProgress

```
root@ip-10-0-1-225:/home/ubuntu# aws elbv2 describe-target-health \
--target-group-arn arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-green-tg/7610a9276362e3dd \
--region ap-south-1 \
--query 'TargetHealthDescriptions[][Target.Id,Target.Port,TargetHealth.State,TargetHealth.Reason]' \
--output table
```

DescribeTargetHealth			
10.0.3.107	80	healthy	None
10.0.3.246	80	draining	Target.DeregistrationInProgress

A target must pass the health check before it is considered healthy and eligible to receive traffic.

5.5 Deploy the New Application Version

After Jenkins pushes a new validated Docker image to Amazon ECR, update the task definition to reference the new image tag.

Update the ECS task definition to reference the newly pushed ECR image.

```
<AWS_ACCOUNT_ID>.dkr.ecr.<AWS_REGION>.amazonaws.com/capstone-app:<IMAGE_TAG>
```

Register the updated task definition.

```
aws ecs register-task-definition \
--cli-input-json file://ecs-task-definition.json \
--region <AWS_REGION>
```

Update the Green service with the new task definition.

```
aws ecs update-service \  
  --cluster capstone-cluster \  
  --service quiz-app-green \  
  --task-definition quiz-app \  
  --region <AWS_REGION>
```

Verify the Green service.

```
aws ecs describe-services \  
  --cluster capstone-cluster \  
  --services quiz-app-green \  
  --region <AWS_REGION>
```

DescribeTargetHealth			
10.0.3.107	80	draining	Target.DeregistrationInProgress
10.0.2.20	80	healthy	None

The Green environment is updated with the new application version while the Blue environment remains available.

5.6 Validate the Green Environment

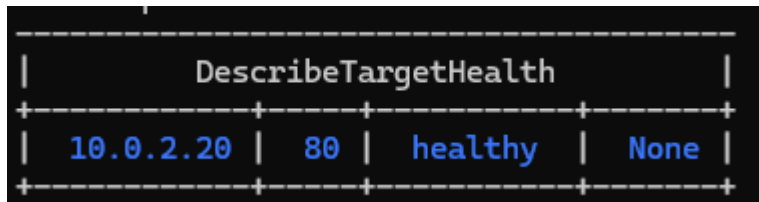
After the Green task starts, verify that the task is running and that the Green target group reports a healthy target.

Check the ECS task.

```
aws ecs describe-tasks \  
  --cluster capstone-cluster \  
  --tasks <TASK_ARN> \  
  --region <AWS_REGION>
```

Check the Green target group.

```
aws elbv2 describe-target-health \  
  --target-group-arn <GREEN_TARGET_GROUP_ARN> \  
  --region <AWS_REGION>
```



DescribeTargetHealth			
10.0.2.20	80	healthy	None

5.7 Shift Traffic from Blue to Green

After the Green environment has been deployed successfully and its health checks have passed, update the Application Load Balancer listener to route traffic to the Green target group.

```
aws elbv2 modify-listener \  
  --listener-arn <LISTENER_ARN> \  
  --default-actions  
  Type=forward,TargetGroupArn=<GREEN_TARGET_GROUP_ARN> \  
  --region <AWS_REGION>
```

Verify the listener configuration.

```
aws elbv2 describe-listeners \  
  --listener-arns <LISTENER_ARN> \  
  --region <AWS_REGION>
```

The traffic flow changes from

ALB → Blue Target Group → Blue ECS Service

to:

ALB → Green Target Group → Green ECS Service

```

"TargetGroups": [
  {
    "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-green-t
62e3dd",
    "Weight": 1
  }
]

```

The Blue environment remains available after the traffic switch, allowing it to be used for rollback if an issue is detected with the new application version.

5.8 Verify Rollback

If an issue is detected after the traffic switch, restore traffic to the Blue environment.

```

aws elbv2 modify-listener \
  --listener-arn <LISTENER_ARN> \
  --default-actions
Type=forward,TargetGroupArn=<BLUE_TARGET_GROUP_ARN> \
  --region <AWS_REGION>

```

Verify the listener.

```

aws elbv2 describe-listeners \
  --listener-arns <LISTENER_ARN> \
  --region <AWS_REGION>

```

The traffic flow is restored from

ALB → Green Target Group → Green ECS Service

to:

ALB → Blue Target Group → Blue ECS Service

This allows the previously running application version to resume serving traffic without immediately removing the Green environment.

```

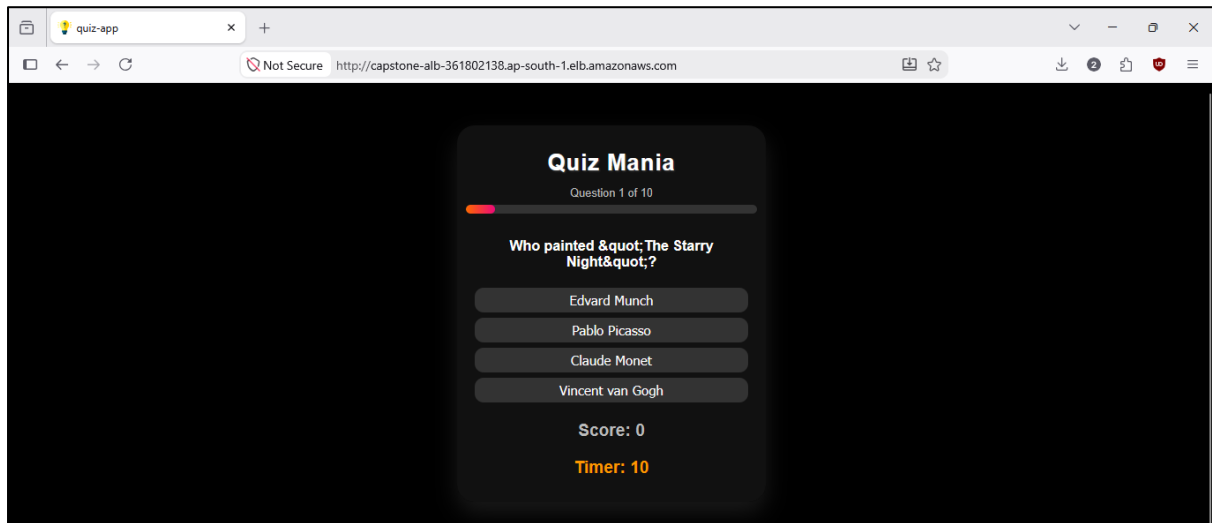
"DefaultActions": [
  {
    "Type": "forward",
    "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:653455484229:targetgroup/quiz-app-blue-tg
ff246",
    "ForwardGroup": "f

```

5.9 Verify the Application through the Load Balancer

Open the ALB DNS name in a browser.

```
http://<ALB_DNS_NAME>
```



Step 6: Monitoring and Observability

The deployed application and AWS infrastructure are monitored using **Amazon CloudWatch and Grafana**. CloudWatch provides application logs and infrastructure metrics, while Grafana uses CloudWatch as its data source to visualize the monitoring data through dashboards.

6.1 Configure CloudWatch Logging

Configure the ECS task to send container logs to **Amazon CloudWatch Logs**. The ECS task definition uses the AWS Logs driver to forward application output to the configured CloudWatch log group.

ecs-task-definition.json

```
{
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/quiz-app",
      "awslogs-region": "<AWS_REGION>",
      "awslogs-stream-prefix": "ecs"
    }
  }
}
```

The ECS container logs are collected under the /ecs/quiz-app log group, allowing application output to be reviewed from the CloudWatch console.

Verify the log group.

```
aws logs describe-log-groups \
  --log-group-name-prefix /ecs/quiz-app \
  --region <AWS_REGION>
```

```
"logGroups": [
  {
    "logGroupName": "/ecs/quiz-app",
    "creationTime": 1790162722898,
    "metricFilterCount": 0,
    "arn": "arn:aws:logs:ap-south-1:653455484229:log-group:/ecs/quiz-app:*",
    "storedBytes": 0,
    "logGroupClass": "STANDARD",
    "logGroupArn": "arn:aws:logs:ap-south-1:653455484229:log-group:/ecs/quiz-app"
  }
]
```

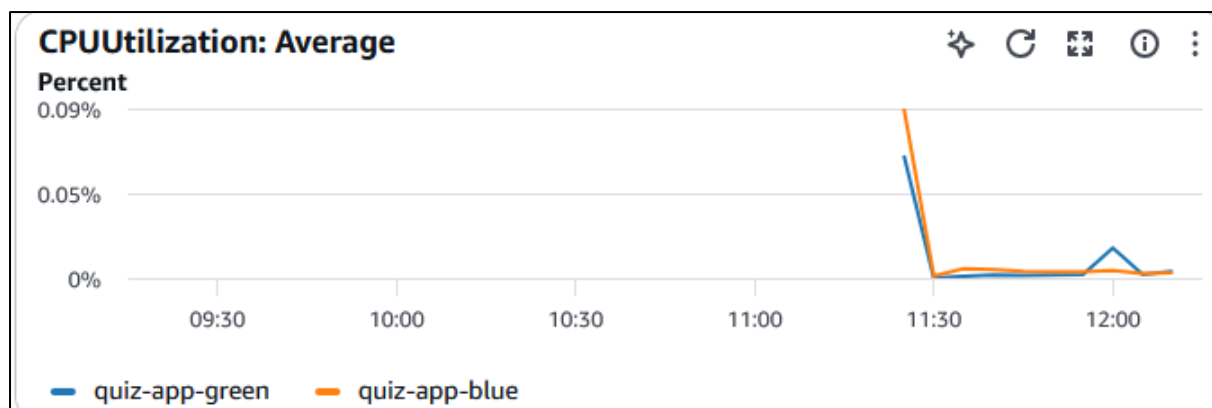
6.2 Configure CloudWatch Metrics

Amazon CloudWatch metrics are used to monitor the health and performance of the ECS application and its supporting AWS resources.

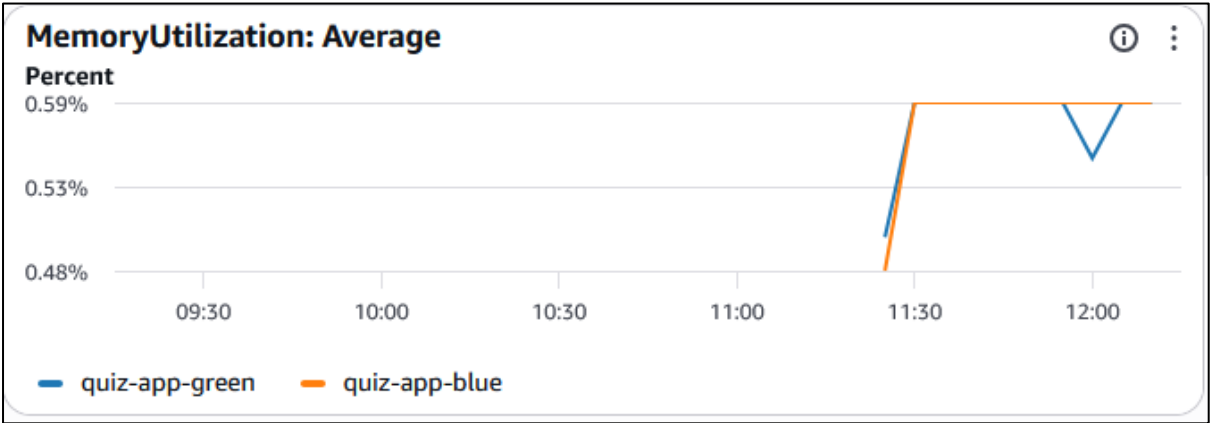
Monitor the following metrics for the **ECS service, running tasks, and Application Load Balancer**.

ECS

CPU Utilization



Memory Utilization



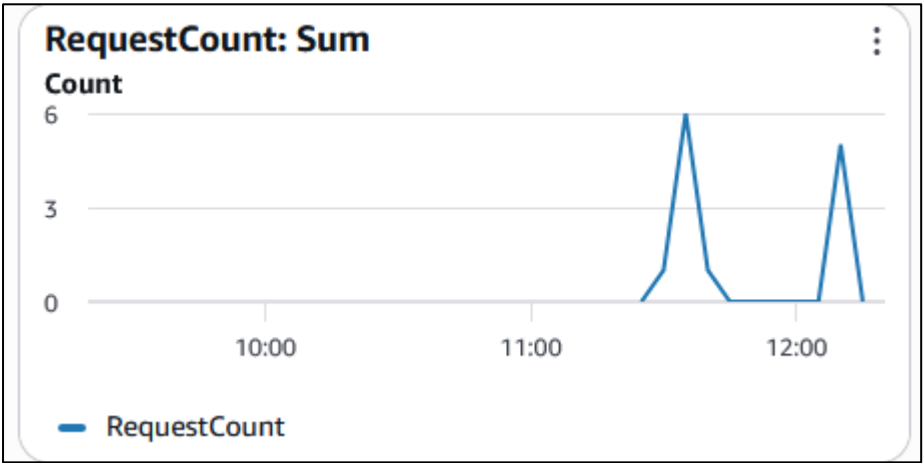
Running Task Count

```
--query 'services[].[serviceName,desiredCount,runningCount]' \
--output table
```

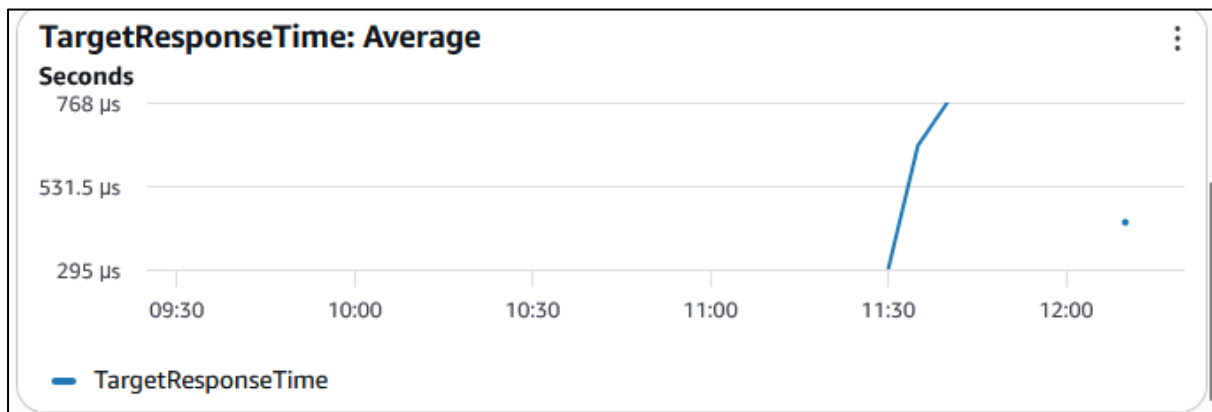
DescribeServices		
quiz-app-blue	1	1
quiz-app-green	1	1

Application Load Balancer

Request Count



Target Response Time



Healthy Host Count and Unhealthy Host Count

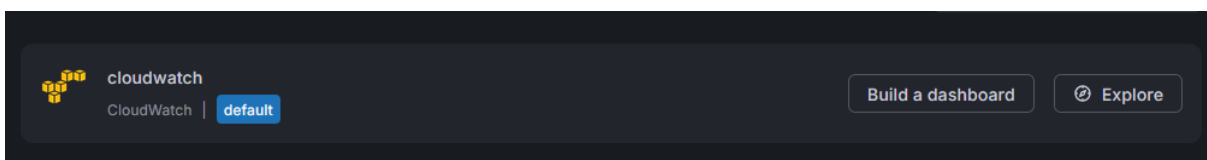
```
=== Blue ===
Healthy   : 1
Unhealthy : 0
=== Green ===
Healthy   : 1
Unhealthy : 0
```

6.3 Configure Grafana

Configure Amazon CloudWatch as the Grafana data source. Grafana retrieves monitoring data directly from CloudWatch and uses it to create dashboards for the ECS services and Application Load Balancer.

In Grafana:

1. Open **Connections / Data Sources**.
2. Select **Add data source**.
3. Select **Amazon CloudWatch**.
4. Configure the AWS region as `<AWS_REGION>`.
5. Configure authentication using the AWS credentials available to the Grafana server.
6. Select **Save & Test**.



The DevOps EC2 instance uses an IAM role with CloudWatch read access, allowing Grafana to retrieve CloudWatch metrics without storing AWS access keys on the server.

6.4 Create Monitoring Dashboard

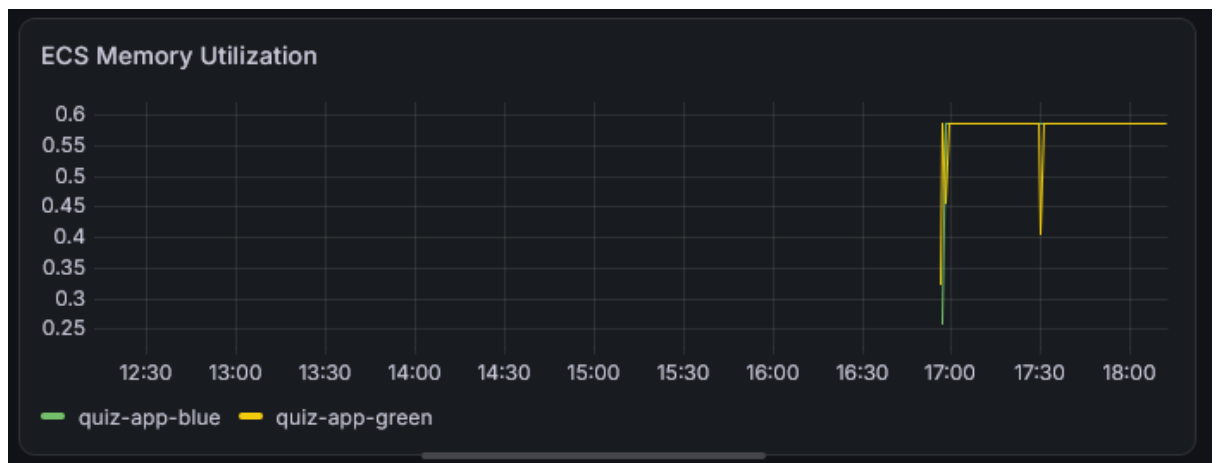
Create a Grafana dashboard using the configured CloudWatch data source.

Configure dashboard panels for the following.

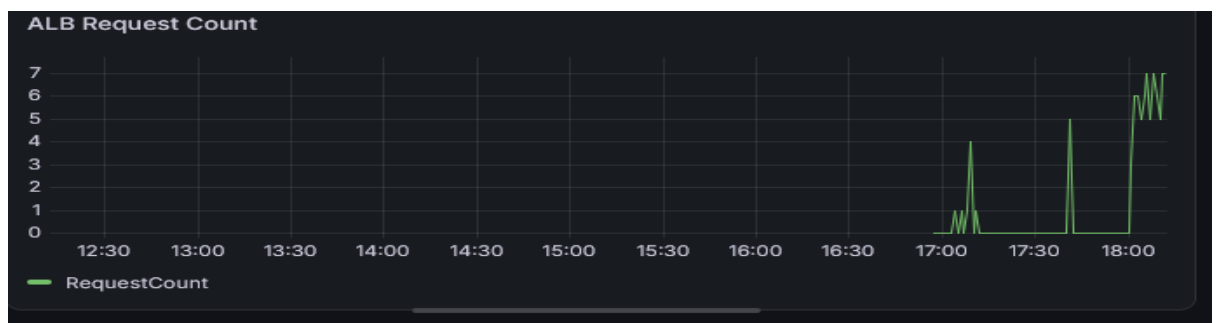
- ECS CPU Utilization



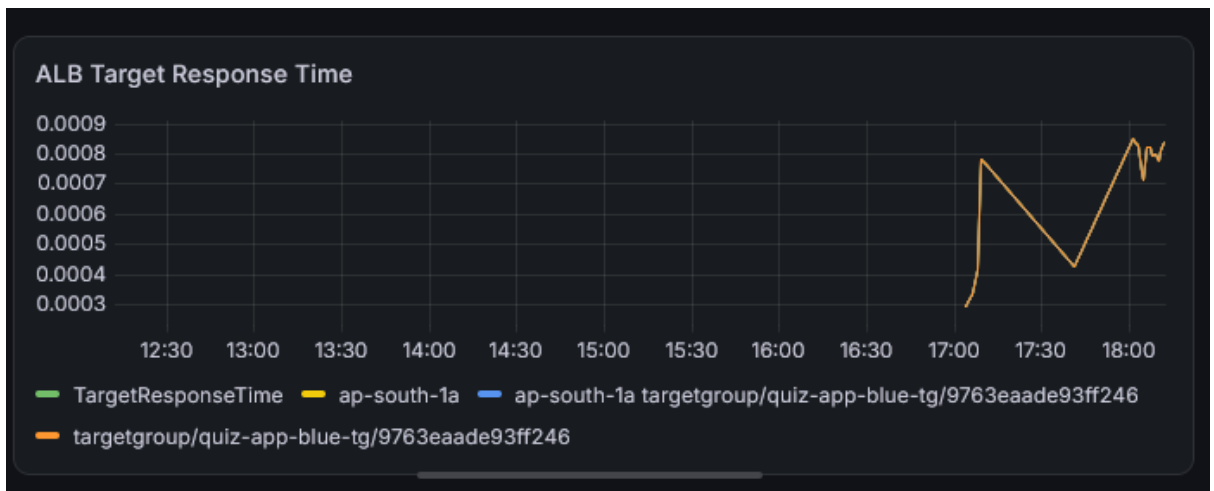
- ECS Memory Utilization



- ALB Request Count



- ALB Target Response Time



- Healthy Host Count



- Unhealthy Host Count



Keep Amazon CloudWatch as the DataSource for everything.

The ECS panels use the AWS/ECS namespace, while the Application Load Balancer panels use the AWS/ApplicationELB namespace.

Conclusion

The project implements an automated application delivery workflow on AWS using **Terraform, Ansible, Jenkins, Docker, Amazon ECR, and Amazon ECS**. Terraform provisions the required AWS infrastructure, while Ansible configures the DevOps server with the required tools. Jenkins automates the application build, linting, code-quality analysis, vulnerability scanning, Docker image creation, and delivery to Amazon ECR.

The application is deployed to **Amazon ECS using a Blue/Green deployment strategy**, with an Application Load Balancer providing controlled traffic switching between the Blue and Green environments. Health checks are used to validate the new application version before traffic is shifted, while the previous environment remains available for rollback.

Amazon CloudWatch provides application logs and infrastructure metrics, while **Grafana** uses CloudWatch as its data source to provide monitoring dashboards. The project brings together infrastructure provisioning, configuration management, CI/CD automation, containerization, security scanning, deployment, and monitoring into a single AWS-based application delivery workflow.